Model Selection for Prediction

Stat 230 - Spring 2026

> **Note**
>
> This is a tutorial: run the code cells in your browser, and record your answers to each question (**A - G**) in your homework write-up. (The packages take a little while to load the first time, and the repeated-splits cell in Part 3 takes a minute or so to run.)

> **Tip**
>
> Run the code chunks in order: objects you create in one chunk (like a fitted model) are available in the chunks that follow.

0.1 Overview

In class, we built a model to *understand* the relationship between ridership, weekends, and holidays. In this activity, you will build models to **predict**: a bike-share company wants to forecast the number of `rides` on days it hasn't seen yet. They do not care about the relative importance or significance of the predictor variables; they only care about how well the model can predict future days.

For prediction, a "good" model is one that does well on *new* data, not one that has the smallest p-values or the best fit to the data we already have.

We will compare six candidate models of increasing complexity. Run the code below to load the packages and the data, and set up the models.

> **Note:**
>
> There is lots of unfamiliar code in this activity! You do not need to understand the nuts and bolts of how the code works, as long as you get the gist of what it's doing from the descriptions.

```
library(dplyr)
library(purrr)
library(broom)
library(rsample)
library(yardstick)
library(parsnip)
library(workflows)
library(tune)

bikes <- read.csv("https://aluby.github.io/stat230/data/bikes.csv")
n <- nrow(bikes)

models <- list(
  M1 = rides ~ temp_actual,
  M2 = rides ~ temp_actual + humidity + windspeed,
  M3 = rides ~ temp_actual + humidity + windspeed + weekend + holiday + year,
  M4 = rides ~ temp_actual + humidity + windspeed + weekend + holiday + year + seasor
  M5 = rides ~ (temp_actual + humidity + windspeed + weekend + year + season)^2,
  M6 = rides ~ (temp_actual + humidity + windspeed + weekend + year + season)^3
)
```

> **What are M5 and M6?**
>
> `(a + b + c)^2` in a formula means "all of the main effects and all of the two-way interactions," and `^3` adds the three-way interactions as well. These models have lots of coefficients (M5 has 34, M6 has 74), so they are very flexible.

The **root mean squared error** (RMSE) measures the typical size of a prediction error, in the units of the response (here, rides):

$$RMSE = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}$$

0.2 Part 1: Judging models by the data used to fit them

Let's start with one model. We fit it to **all** of the data, use `augment()` to get the fitted values, and use `rmse()` from the {yardstick} package to compute the RMSE of the fitted values.

```
m1 <- lm(rides ~ temp_actual, data = bikes)
m1_aug <- augment(m1)
rmse(m1_aug, truth = rides, estimate = .fitted)
```

Now do the same thing for all six models:

```
bikes_fits <- map(models, \(f) lm(f, data = bikes))

tibble(
  model = names(models),
  k = map_int(bikes_fits, \(m) length(coef(m))),
  rmse = map_dbl(bikes_fits, \(m) rmse(augment(m), truth = rides, estimate = .fitted)
)
```

A. What happens to the RMSE as the models get more complex? Would you trust this table to tell you which model will predict *new* days best? Why or why not?

0.3 Part 2: Training and test data

To find out how well a model predicts new data, we **hold some data back**. We fit ("train") the model on a *training set* and measure its prediction error on a *test set* using data that the model has never seen.

Here we randomly use 80% of the days as training data and the remaining 20% as test data.

```
set.seed(75)

bikes_split <- initial_split(bikes, prop = 0.8)
bikes_split

bikes_train <- training(bikes_split)
bikes_test  <- testing(bikes_split)
```

B. Fill in the blanks so that each model is **fit on the training data** and then used to predict the **test data**. The code computes the RMSE on both datasets for every model. (For new data, `augment()` takes a new`data` argument.) Record the results.

```
train_test_rmse <- map_dfr(models, \(f) {
  m <- lm(f, data = bikes_train)                        # fit on the TRAINING data
  tibble(
    train = rmse(augment(m), truth = rides, estimate = .fitted)$.estimate,
    test  = rmse(augment(m, newdata = bikes_test), truth = rides, estimate = .fitted)
  )
}, .id = "model")

train_test_rmse
```

C. Answer the following using your table.

a. As the models get more complex, what happens to the **training** RMSE? What happens to the **test** RMSE?
b. Which model has the smallest test RMSE?
c. When a model is **overfit**, it fits the data used to build it very well but predicts new data poorly. Which model(s) appear to be overfit? Which appear to be **underfit** (too simple to capture the patterns in the data)? Explain using the gap between the training and test RMSE.

0.4 Part 3: These results depend on the random seed

The results above depend on which days happened to land in the test set. Since we chose our 80/20 train/test split based on a random sample, if we had a *different* random sample, we would get different results. Edit the code below to use a different `set.seed` value and then re-generate the RMSE table.

```
set.seed(230)
bikes_split <- initial_split(bikes, prop = 0.8)
bikes_split

bikes_train <- training(bikes_split)
bikes_test  <- testing(bikes_split)

train_test_rmse <- map_dfr(models, \(f) {
  m <- lm(f, data = bikes_train)
  tibble(
    train = rmse(augment(m), truth = rides, estimate = .fitted)$.estimate,
```

```
    test  = rmse(augment(m, newdata = bikes_test), truth = rides, estimate = .fitted)
  )
}, .id = "model")

train_test_rmse
```

D. Report the *seed you used* and your new *RMSE table*. Which model has the smallest RMSE on the training set? What about the test set?

E. Is your answer to Q3(b) (from the first split) the same as your answer to Q4? What does this tell you about using a single train/test split to choose a model?

A more efficient version of repeated splitting is **5-fold cross-validation**: randomly split the *training* data into 5 equal "folds." Each fold takes a turn as the held-out set while the model is fit on the other four folds, so every training day gets predicted exactly once. Model selection uses *only the training data*, so that the test set stays untouched. We will not cover this in Stat230, but if it sounds interesting, it is a topic in Stat270 offered this winter.

0.5 Part 4: Compare to AIC and BIC

AIC and BIC don't use any held-out data: they estimate a model's quality from the data it was fit to, with a penalty for complexity. Since our goal is prediction, we compute them using the **training data only**, and use `glance()` to pull out the criteria.

```
ic <- map_dfr(models, \(f) {
  glance(lm(f, data = bikes_train)) |>
    select(adj.r.squared, AIC, BIC)
}, .id = "model")

ic
```

F. Which model does AIC prefer? Which does BIC prefer? How does this compare with the training/test set approach?

G. Suppose two different people use this data:

- A bike-share company wants to **forecast** next week's ridership to decide how many bikes to put out.
- A city planner wants to **understand** how weekends and holidays affect ridership, after accounting for the weather.

Which of the six models (or what kind of model) would you recommend to each person? Do they necessarily need the same model? Explain, referring to your results above.

0.6 Function quick reference

Function	Purpose
`set.seed()`	Make random choices reproducible
`initial_split()`	Randomly split data into training and test sets
`training()`, `testing()`	Extract the training or test data from a split
`lm(formula, data)`	Fit a linear model
`augment(model, newdata)`	Add fitted values or predictions (`.fitted`) to a data frame
`rmse(data, truth, estimate)`	Root mean squared error
`glance()`	One-row summary of a model, including adjusted R^2, AIC, and BIC
`mc_cv()`, `vfold_cv()`	Create repeated random splits or v-fold cross-validation folds
`workflow()`, `add_model()`, `add_formula()`	Bundle a model type and a formula together
`fit_resamples()`, `collect_metrics()`	Fit a workflow to each split or fold and summarize the error
`map()`, `map_dfr()`	Repeat a computation for each model in a list